

Análisis – Caso de Detectives

1. ¿Qué propiedades y métodos tendrá cada clase?

Para resolver este problema, planteamos las siguientes cuatro clases y sus funciones: Caso, Locación, Pista y Main.

Caso: La función de esta clase es representar una investigación de la agencia de detectives. Contiene la información general, las listas de ubicaciones y pistas relacionadas.

Atributos:

- Nombre
- Código
- DetectiveResponsable
- Ubicaciones: Basic Array (tamaño máximo 5) de objetos Location;
- Pistas: ArrayList de objetos.

Métodos:

- Constructor: Inicialización principal de atributos.
- Getters para los atributos: Acceso a los datos.
- Métodos para agregar ubicaciones al array con su ubicación determinada.
- GetUbicaciones para devolver todas las ubicaciones.
- GetLocation para devolver una ubicación en particular.
- Modificador del nivel de riesgo y estado de una ubicación.
- RemoveLocation para eliminar la ubicación.
- Añadir pistas a un arrayList.
- GetAllClues para consultar todas las pistas actuales.
- Buscador de una pista por código.
- Modificador de una pista.
- RemoveClue para eliminar una pista.
- Métodos para calcular los datos que se presentarán en el informe del investigador, según se solicite.

Además, la clase Caso sería el controlador de los métodos para las colecciones, gestionando las validaciones para ubicaciones (posiciones inválidas, nulos) y pistas (códigos duplicados).

Ubicación: Representa un lugar que debe investigarse como parte del caso.

Atributos:

- Código: String.
- Nombre: String.
- Dirección: String.
- nivelDeRiesgo: Int (1 al 10).
- Estado: String.

Métodos principales:

- Constructor: Inicialización principal de atributos.
- Métodos get para acceder a sus atributos
- Métodos set para modificar los atributos que pueden cambiar
- Método para modificar el nivel de riesgo
- Método para modificar el estado
- Método toString() para representar la información de la ubicación

El nivel de riesgo debe validarse tanto al crear una ubicación como al modificarla. Locación solo permitirá valores entre 1 y 10; en caso contrario, se generará una `IllegalArgumentException`.

Pista: La clase Pista será una representación de la evidencia encontrada durante la investigación.

Atributos:

- Código: String.
- descripción: String.
- tipoDeEvidencia: String.
- nivelDeImportancia: Float (entre 1 y 10).
- nivelDeConfiabilidad: Float (entre 0 y 100).

Métodos principales:

- Constructor: Inicialización principal de atributos.
- Getters para obtener los valores de las variables
- Setters para actualizar las variables (con la excepción del código de la pista)
- un método `toString()` para mostrar la información de la pista.

Al crear o modificar una pista, sus niveles de importancia y confiabilidad deben cumplir los siguientes requisitos: entre 1 y 10 para el primer valor, y entre 0 y 100 para el segundo. Cualquier otra cosa dará como resultado una `IllegalArgumentException`

Main: Sería la clase principal del sistema al contener el método `main()` con el cual tendrá que arrancar el programa, capturar los datos con la clase Scanner, mostrar el menú y llamar a los métodos de las clases necesarias. Además, también deberá gestionar las excepciones que surjan, por ejemplo, la entrada del usuario por teclado y el tratamiento de errores como el `InputMismatchException`.

2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Clase	Propiedad	Tipo
Caso	nombre	String
Caso	código	String
Caso	detectiveResponsable	String
Caso	detectiveResponsable	Ubicación[]
Caso	pistas	ArrayList
Ubicación	código	String
Ubicación	nombre	String
Ubicación	dirección	String
Ubicación	nivelRiesgo	int
Ubicación	estado	String
Pista	código	String
Pista	descripción	String
Pista	tipoEvidencia	String
Pista	nivelImportancia	int
Pista	nivelConfiabilidad	int

Se podrá utilizar void para los métodos, en caso de que no devuelvan ningún valor. Los métodos de consulta o búsqueda requerirán el tipo de dato adecuado para devolver el resultado esperado. Así, una búsqueda de pista requerirá un objeto del tipo Pista, mientras que una consulta a una página requerirá un objeto Ubicación. Los métodos que realicen cálculos requerirán valores numéricos (int o double) de acuerdo con la expresión utilizada para el cálculo.

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad ubicaciones de la clase Caso se implementará mediante un arreglo básico de objetos:

Ubicacion[]

El arreglo tendrá un tamaño fijo de 5 elementos, ya que un caso solo puede tener un máximo de cinco ubicaciones de investigación. Las posiciones sin ubicación tendrán un valor nulo. Por lo tanto, es necesario comprobar previamente que no sean nulas antes de acceder a los métodos de un objeto almacenado en esa posición. Por ejemplo:

ubicaciones[0] == nulo --- indica que la primera posición está disponible.

El uso de la matriz permite practicar el almacenamiento de una cantidad fija de objetos, el acceso mediante índices, los recorridos y la comprobación de que ninguna matriz sea nula, tal y como pide el trabajo de laboratorio.

4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad «Pistas» de la clase «Caso» se implementará utilizando el ArrayList. La razón por la que se utiliza un ArrayList es que el número de pistas que se pueden encontrar durante una investigación es desconocido y, muy probablemente, variable a lo largo de la ejecución del programa.

De este modo, la aplicación podrá añadir y eliminar nuevas pistas, examinarlas para buscar indicios, modificar su contenido y borrar aquellas que resulten irrelevante o falsas. El caso práctico también indica que debe utilizarse directamente el ArrayList y que no debe aplicarse la interfaz List.

5 ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?

Con el fin de mantener el encapsulamiento, es decir, ocultar los atributos y las operaciones de una clase a otras clases, debemos declarar como privados esos atributos y operaciones de la clase de la que forman parte. Por otro lado, los métodos que forman parte de la interfaz pública de dicha clase (es decir, los «métodos» del «modelo»), como por ejemplo los métodos get y set, los constructores y, en general, aquellas operaciones que puedan dar acceso a los atributos privados utilizando operaciones get y set, deben ser, en el mejor de los casos, públicos. En otras palabras, los campos privados de una clase no deben ser accesibles desde otros objetos; en cambio, deben consultarse y modificarse mediante métodos, lo cual está controlado por los modificadores de acceso.

En Main.java, el método main() debe ser público, estático y void, ya que Main es la clase inicial del programa. Esto permite hacer seguimiento de estos modificadores de acceso y garantizar que los campos privados no puedan acceder directamente desde otras clases.

6. ¿Qué parámetros serán requeridos por los métodos en sus clases?

Los parámetros dependerán de la operación deseada. A continuación, se muestran los parámetros necesarios:

Métodos relacionados con las ubicaciones:

- La posición del arreglo.
- El código.
- El nombre.
- La dirección o descripción.
- El nivel de riesgo.
- El estado.

Se necesita una ubicación para realizar una búsqueda, y además, la posición en el arreglo.

Para modificar una ubicación será necesario:

- La posición.
- El nuevo nivel de riesgo.
- Un nuevo estado.

Para descartar una ubicación será necesaria:

- La posición de la ubicación dentro del arreglo.

Para registrar una pista se necesitarán los siguientes datos:

- Código.
- Descripción.
- Tipo de evidencia.
- Nivel de importancia.
- Nivel de confiabilidad.

Para buscar, modificar o eliminar una pista será necesario:

- El código de la pista.

Los métodos de consulta general y los que llevan a cabo los cálculos de generación del informe no necesitan recibir información adicional, ya que pueden trabajar directamente con los datos almacenados en el objeto Caso.

7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales de los objetos «Caso», «Ubicación» y «Pista» se establecerán en sus respectivos constructores.

A la hora de crear una ubicación, se debe comprobar que el nivel de riesgo se sitúa entre 1 y 10. Lo mismo se aplica cuando se modifica la ubicación.

En el caso de una pista, hay que asegurarse de que su nivel de importancia se encuentre entre 1 y 10, y que el nivel de confiabilidad sea superior a 0 y no supere el 100. Estas dos condiciones también deben cumplirse al modificar la información de una pista. Si alguno de estos valores no cumple los requisitos, se lanzará una *IllegalArgumentException*.

El código de las pistas no puede superponerse a otro código utilizado previamente para otra pista. Por lo tanto, debe comprobarse que no haya dos pistas con el mismo código antes de introducir una nueva.

Al crear un nuevo caso, no se han registrado ubicaciones en el array de ubicaciones y no hay pistas en el ArrayList.

8. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?

Para determinar si una posición contiene una ubicación hay que acceder a la posición mediante su índice y comprobar si la referencia es nula.

Si: `ubicaciones[ posición ] == nulo`

la posición estará disponible y no tendrá una ubicación registrada.

Si la referencia no es nula, la posición contiene una instancia de Ubicación .

La comprobación debe llevarse a cabo antes de intentar utilizar métodos de una ubicación para evitar que se produzcan errores debidos a intentar utilizar una referencia inexistente. También es necesario comprobar que el índice se encuentre dentro de los límites del *array* antes de acceder a él.

9. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?

Las operaciones sobre las pistas se llevarán a cabo recorriendo la ArrayList de objetos Pista. Al buscar una pista, recorreremos todos los elementos de la colección y compararemos el código de cada pista con el código facilitado por el usuario. En el momento en que encontramos una coincidencia, habremos encontrado la pista deseada.

Para modificar una pista, en primer lugar, la buscaremos por código, y una vez encontrada, modificaremos sus atributos llamando a los métodos correspondientes y revalidaremos la importancia y la confiabilidad. Al eliminar una pista, en primer lugar, la localizaremos por código y, una vez encontrada, la eliminaremos del ArrayList.

Si no encontramos ninguna pista con el código introducido, debemos indicarlo al usuario sin cerrar el programa. Todas estas operaciones deben realizarse únicamente con los elementos del ArrayList.

10. ¿Qué situaciones del programa pueden producir excepciones?

A lo largo del programa, durante la ejecución del mismo, pueden surgir numerosas circunstancias que den lugar a errores y que deben tenerse en cuenta y gestionarse.

inputMismatchException: Es posible que se produzca cuando el programa solicita información por teclado (mediante Scanner) y este no coincide con lo esperado (por ejemplo, se introduce texto cuando se espera un número). Como solución, habría que implementar una sentencia try-catch (limpiar la cola de entrada, la siguiente lectura de la línea de comandos será la siguiente línea, no el resto de la línea actual). Por lo tanto, esta excepción debería gestionarse también (de nuevo, limpiar la cola de entrada).

Excepción de argumento ilegal: En cuanto a las excepciones, esto se lanzará cuando se proporcionen argumentos inválidos a un método que requiera valores numéricos de un objeto. Por ejemplo:

- Al intentar crear una ubicación con un nivel de riesgo inferior a 1 o superior a 10.
- Al modificar el riesgo de una ubicación de forma inválida.
- Al proporcionar un nivel de importancia para una pista que no es igual a 0 o no es un número entre 1 y 10.
- Al proporcionar un nivel de credibilidad (porcentaje) para una pista que no es igual a 0 o a 100.

Índices del array fuera de rango: El usuario puede proporcionar índices para las posiciones que no existen (menores que 0 o mayores que el tamaño máximo del array dado). Del mismo modo, es posible que el usuario indique la posición del array que contiene una referencia nula. Esta situación puede evitarse comprobando que cada posición es correcta (dentro del rango) y que la referencia no es nula.

Operaciones sobre listas vacías: El usuario intenta acceder a elementos de una lista vacía, por ejemplo, leer, borrar, modificar pistas, etc. El programa debe gestionar estas situaciones, comprobar si la lista está vacía e impedir que se modifique cuando esté vacía.

Finalmente, El programa debe incluir al menos una vez la sentencia finalmente (cuando tenga sentido). Si se da alguna de las situaciones anteriores, el programa debería continuar (para cumplir el requisito del trabajo de laboratorio).

Diagrama UML:

Diagrama UML – Sistema de Investigación de Casos

